CHAPTER 1

CONCLUDING THE BUILDING BLOCKS DISCUSSION

1 AI Evaluation of Building Blocks in E-Commerce Platform 2
2 AI Evaluation of Building Blocks in Online Education System 3
3 Wrapping Up the Building Blocks Discussion 4
4 The RESHADED Approach for System Design 5

Wrap up the discussion on building blocks, evaluate your understanding using AI, and learn the RESHADED approach to tackle unseen System Design problems.

1 AI Evaluation of Building Blocks in E-Commerce Platform

In this lesson, we will assess our understanding of building blocks using an AI interactive widget. Let's begin!

1.1 An e-commerce platform

Consider an abstract and simplified system design for an e-commerce platform outlined below. The design comprises various components that collaborate to execute diverse tasks. The customer engages with multiple services such as search, shopping cart, and order-taking services. While details for each component and service in the design have been provided, some missing components are not described. Your objective is to identify these missing components and concisely describe their functions using the AI interactive widget given at the end of the lesson.

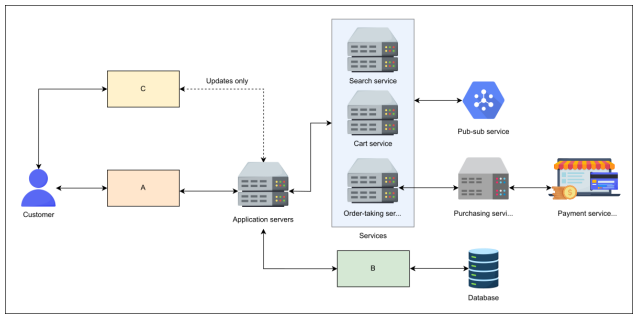


Figure 1.1: A high-level design of the e-commerce platform

Different components in the design are described below:

1. **Application servers:** They provide middleware services by facilitating communication between different components of an application. They also forward user queries to the respective services.
2. **Search service:** This component handles search queries requested by users.
3. **Cart service:** This service handles the shopping cart where the customers select different products to purchase.
4. **Order-taking service:** This service initiates customer orders, triggering the purchasing service upon the user's request.
5. **Payment service provider (PSP):** The PSP handles payment transactions securely, interacting with external payment gateways.
6. **Database:** It stores the user and product information, such as images, prices, descriptions, and other metadata permanently.

Your task is to determine the missing components A, B, and C in the above design. You need to provide your answer in the correct order (A, B, and then C) along with a concise description.

Question:

Find the missing building blocks in the e-commerce system design.

Reference Answer:

Component A = load balancers that distribute incoming traffic across multiple servers. Component B = the caching layer, which optimizes performance by storing frequently accessed data. It can also be referred to as a distributed cache. Component C = a Content Delivery Network (CDN) used for optimizing the delivery of static assets like images and stylesheets. It also enhances website performance by serving static content from strategically distributed servers. CDNs get updated content from the origin servers, which, in this case, are application servers.

2 AI Evaluation of Building Blocks in Online Education System

2.1 Online education system

The following is a high-level system design of an online education system that provides different types of video tutorials for various subjects. Users opt for courses to buy and watch video tutorials. Take a look at the architecture to answer the questions provided below. The details of different types of services included in the design are also given.

In the system, each service is described below:

- **Application servers:** The application servers receive user requests and forward them to the relevant services. Their main purpose is to host, manage, and execute applications, execute user requests, and manage connections with databases and other services.
- **Search service:** This service helps users to find relevant courses on the platform.
- **Shopping cart service:** This service is responsible for keeping the courses that are selected by the users for purchase.
- **Order management service:** This service is responsible for the entire process, from order placement to order fulfillment. It includes tasks like validating orders, processing payments, handling returns and refunds, and offering customer support.
- **Recommendation service:** This service provides personalized content recommendations to users based on their preferences, behavior, and learning goals.
- **Video processing service:** This service handles the tasks of video transcoding, uploading, and on-demand streaming for video tutorials. Its primary users are the administrators who rely on it to upload educational videos.
- **Content delivery network (CDN):** The CDN distributes web content consisting of HTML pages, JavaScript files, images, videos, and other media files to users in various geographic locations.
- **Pub-sub service:** The pub-sub service decouples different services and enables asynchronous communication between them.

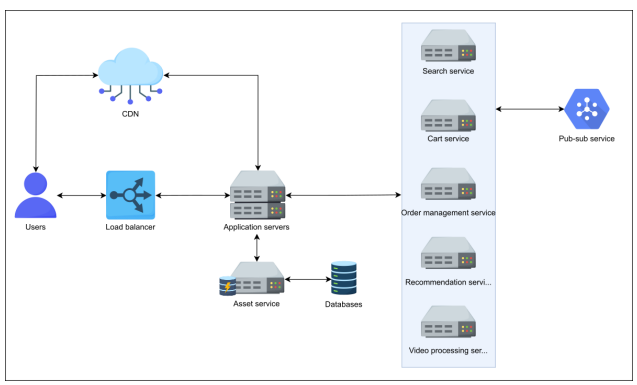


Figure 1.2: An abstract system design of an online education system

As we can see, the given design is simplified, but there are certain issues that the system faces, which we've listed below. In this exercise, your task is to identify the building blocks that, if incorporated, can remove these problems for optimal operation of the system. Here are the problems the system is currently facing:

1. If certain users continuously send an excessive number of requests (probably a denial of service attack), the system could potentially overwhelm our back-end servers. It 's imperative to mitigate such request surges to prevent any adverse impact on the server infrastructure. Which component do you think can help fix this issue?

2. Suppose we're managing a user base of 200 million, and they're generating a massive volume of requests across various categories, for example, video uploads, comments, forum discussions, and module additions. Managing and distinguishing these activities within the system can be a challenge. To efficiently address this issue and uniquely identify each request, which component or building block is needed?

3. The engineering team currently invests a significant amount of time in identifying and troubleshooting system failures. To streamline this process, it's crucial to integrate an appropriate building block or service into the system design. What component do you recommend for identifying and troubleshooting failures in the system?

4. Besides the drawbacks, we want to add a feature to allow users to see the number of views on each video tutorial. Essentially , we need a counter to keep a count of the large number of views on a video. What approach would you propose to efficiently estimate the approximate number of views in this high-traffic environment?

Note: Please provide answers in the same order as that of the questions.

Question:

Improve the system design of an online education system.

Reference Answer:

The answers to the above questions are given below in the same sequence as that of the #Questions:

1. To manage and mitigate the impact on backend servers due to a vast number of incoming requests, we should incorporate a rate limiter with a suitable rate-limiting algorithm.

2. To assign unique identifiers to different requests and events occurring within the system, we should incorporate the sequencer building block in the system.

3. The system should encompass a monitoring service to reduce the time required for problem identification and troubleshooting.

4. To efficiently estimate the number of views in high-traffic environments, we need a building block known as a sharded counter.

3 Wrapping Up the Building Blocks Discussion

We have covered a good number of building blocks that will enable us to solve bigger design problems. In the coming chapters, we'll solve a variety of problems that will use various building blocks.

Since we've already covered the building blocks, we can focus on solving bigger design problems. From this point onwards, we'll assume that you're well acquainted with all the discussed building blocks when we use them in our coming design problems.

3.1 What's next?

We'll learn and explore sixteen design problems in the next chapters. We'll study the following systems:

1. YouTube

2. Quora

3. Google Maps
4. Yelp
5. Uber
6. Twitter
7. Newsfeed System
8. Instagram
9. TinyURL
10. Web Crawler
11. WhatsApp
12. Typeahead
13. Google Docs
14. Deployment System
15. Payment System
16. ChatGPT System

Each of these designs is an independent chapter; however, we recommend that you proceed through them in the assigned order. Some problems have background material that is helpful in other design problems. For example, working through the Google Maps problem first can be useful for readers interested in learning about the Uber design problem.
We'll conclude our course with the "Spectacular Failures" chapter, following some (optional) chapters on additional resources. In the section on spectacular failures, we'll discuss how a minor bug or mistake led to a significant outage or failure of some of the most successful systems ever designed.

4 The RESHADED Approach for System Design

4.1 Introduction

System design problems are not straightforward. We don't have a universal formula that we can use for all design problems. However, we can use a high-level common strategy to set the tone for a good solution to any design problem. We call this the **RESHADED** approach. Generally, all our design problems are solved by keeping this strategy in mind. RESHADED is a guideline that we'll use to resolve different design problems. Although there's no such thing as a one-size-fits-all solution, using this approach will have its advantages, as we will see.

Breakdown of each letter in the RESHADED approach

[Interactive Mind Map - See online version for interactive visualization]

- **RESHADED**
  - **Requirements**

- **Estimation**
- **Storage schema (optional)**
- **High-level design**
- **API design**
- **Detailed design**
- **Evaluation**
- **Distinctive component/feature**

4.2 Advantages of RESHADED

Before we go through each word in RESHADED, let's cover the benefits of our overall approach. Some of the key advantages of this approach are below:

1. The RESHADED approach helps us remember some key steps for the resolution of every design problem. This means that, at any point in time, there will always be a next step laid out ahead of us.
2. The solution we come up with will have all the basic ingredients required to solve any design problem. Not only that, but the solution offered through the RESHADED approach will be systematic and thoughtful.

4.3 Exploring RESHADED

Below, we describe what steps we take under each word in RESHADED:
Requirements: During this step, we gather all the requirements of the design problem and define its scope. Requirements include understanding what the service is, how it works, and what its main features are. Our goal in this step is to gather the functional and non-functional requirements of a service we are about to design.

Estimation: As the name suggests, this step estimates the resources required to provide the service to a defined number of users. By resources, we mean the hardware or infrastructural resources.

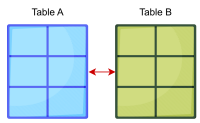
Some sample estimation questions are the following:

1. How many servers will we require to provide smooth services to 500 million daily active users (DAU)

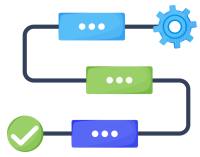
2. How much storage do we need if we have to store 125 million tweets per day, and 20% of tweets contain media?

Estimations are important because they help us understand the scale of the system we'll design. We 'll make key decisions based on the estimate. For example, we'll decide what type of database to use for storing our data, which data structure will give optimal performance, and so on.

Storage schema (optional): This step involves articulating our data model—that is, we define which tables we need and what type of fields are part of each table. However, this is an optional step, and we may not exercise this effort in every design problem.

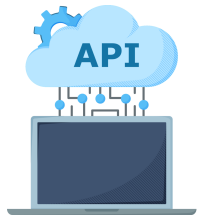


High-level design: This step involves identifying the main components and building blocks we'll use to design our desired system. We do this by getting inspiration from our functional and non-functional requirements.



This is considered the first step toward the complete design of our system and therefore requires further iteration and improvement. Primarily , this section will focus on fulfilling the functional requirements.

API design: The goal in this phase is to build interfaces for our service. Using these interfaces, users can call various services within our system. These interfaces are in the form of API calls and are generally a translation of our functional requirements.

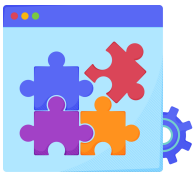


Detailed design: The detailed design starts by recognizing the limitations of the high-level design. We'll capitalize on these limitations to evolve our design. During this step, we'll finalize our design by mentioning all the components and building blocks that we'll use.

We also define the workflow of our design and its usage of different technologies. The detailed design aims to fulfill the functional as well as non-functional requirements of the problem.

Evaluation: This step will measure the effectiveness of our solution. In other words, we justify how our design fulfills the functional and non-functional requirements.

We discuss different trade-offs we made in our solution and also identify room for improvement.

Distinctive component/feature: At the start of this lesson, we discussed how no one-size-fits-all solution exists. This step is to identify a unique aspect for each design problem and discuss it. For example, the Uber design problem has payment service and fraud detection as its unique feature. In contrast, Google Docs has concurrency control, which is required when different users want to edit the same section of a document simultaneously.

Next, we'll apply our guideline (RESHADED) to many design problems.